1. Project question addressed by this pipeline
Our question is if there are any trends from out analysis that we can use to determine an accurate price prediction. Our analysis_text.sql and analysis_parquet.sql pipelines addresses the given question.

2. How result contributes to final project
The results help contribute to the final project by helpin gus calculate the monthly stock-price increase counts, total comparable records, and ratio of increases.

3. Raw input file, delimiter, header status, and fields used
Raw input file: TMCV_minute.tsv
Delimiter: Tabs
Header status:
name date open high low close volume price_change volume_change
Field used: name, date, open, high, low, close, volume, price_change, volume_change

4. Expected schema of analysis-ready intermediate table
Expected to be a .tsv file with a header row and four columns:

Column name	Data type	Description
symbol	VARCHAR	Stock ticker symbol
timestamp	VARCHAR	Date and time of stock record
close	DOUBLE	Closing stock price
price_change	VARCHAR	Price movement (Increased/Decreased)

5. Output schemas for shell, DuckDB-text, and DuckDB-Parquet
All three output schemas are expected to be .tsv files with the same four columns being used:

Column name	Data type	Description
month	Integer	Month number (1-12)
increase_count	Integer	Number of records that increased that month
compared_rows	Integer	Total records that increased or decreased during the month
increase_ratio	Decimal format	increase_count / compared_rows, four decimal place

6. Rules
Filtering: Only rows where the price_change has increased or decreased are included.
Missing-value: None rows don't contribute to the analysis stages but are accepted as a valid price-change in the preparation stage.
Grouping: The scripts extracts month from timestamp, group records from month number, and calculate the increase count, comparable row count, and increase ratio.
Ordering: DuckDB sorts months in ascending order. Shell script loops through months in ascending order and ignores months with no comparable records.
Ranking: Contains monthly aggregates instead of ranked individual records
Tie-handling: Same as ranking.

7. Data-quality or parsing risks
a. If the input delimiter was commas instead of tabs, this would be a mismatch between header and delimiter which could prevent the script from processing.

b. An incomplete timestamp could pass the script's checks since it only checks timestamp format and month and not every part of the date and time.

8. Invariant or expectation used to verify each major stage
For input validation we check that every input data row has exactly nine tab-separated fields. Script aborts otherwise.
For data preparation we check if every output row has the same four fields along with the number of valid rows + the rejected rows must be equal to the total number of input data rows.
For shell analysis, output must be nonempty and its result header should contain month, increase_count, compared_rows, and increase_ratio.
For DuckDB text analysis, output should contain the same monthly aggregate results as the shell analysis.
Parquet file must be readable and should produce the same results as the DuckDB text analysis.

9. Meaningful partitioning rule used by the scheduled jobs
The pipeline should partition the input data by stock symbols so each scheduled job processes records for a specific symbol. Has not yet been implemented in the current scheduled job scripts.

10. Output that can be reused
We could reuse the outputs from the shell to compare the results against the DuckDB results.

Stage Map:
Raw input
 -> Validate and normalize input
  -> Analysis-read TSV
   -> Shell-text analysis
   -> DuckDB-text analysis
    -> DuckDB-text results
     -> Build Parquet file
      -> DuckDB-Parquet analysis
       -> DuckDB-Parquet results
    -> Equivalance check and timing evidence
     -> One scheduled partition per day (not implemented yet)
      -> Final project analysis artifact